

ENSF 380

TOPIC 26: DATABASES PART II

Reminders

Relational databases represent data using tables (relations), rows (tuples), and columns (attributes)

SQL is a language used to manage and retrieve data in relational databases

Applications interact with a database by generating queries and transactions

Relational Databases

Columns

Name	Type
id	integer
name	varchar
owner	varchar
birth	date
death	date

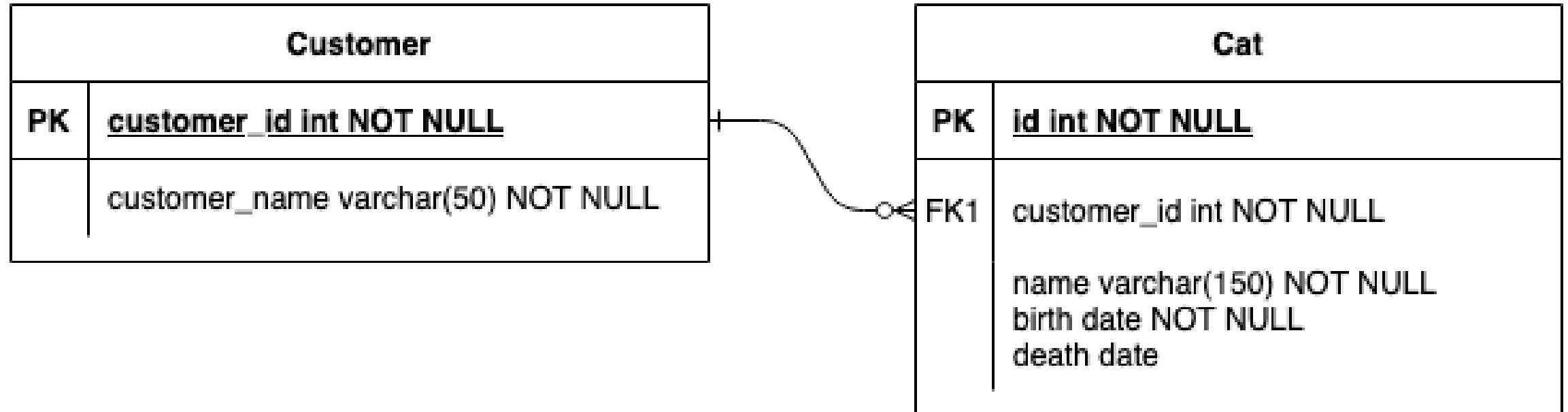
Row

8	Tanka	Crystal	2023-07-07	NULL
----------	--------------	----------------	-------------------	-------------

Primary key

The diagram illustrates a primary key relationship. A red arrow originates from the text 'Primary key' and points to the 'id' column header in the columns table. Another red arrow originates from the same point and points to the first cell of the first row in the rows table, which contains the value '8'. This indicates that the 'id' column is the primary key for the table.

Relational Databases



Relations Versus Objects

Relational Databases

Customer

id	firstname	surname
2	Karl	Kaufmann

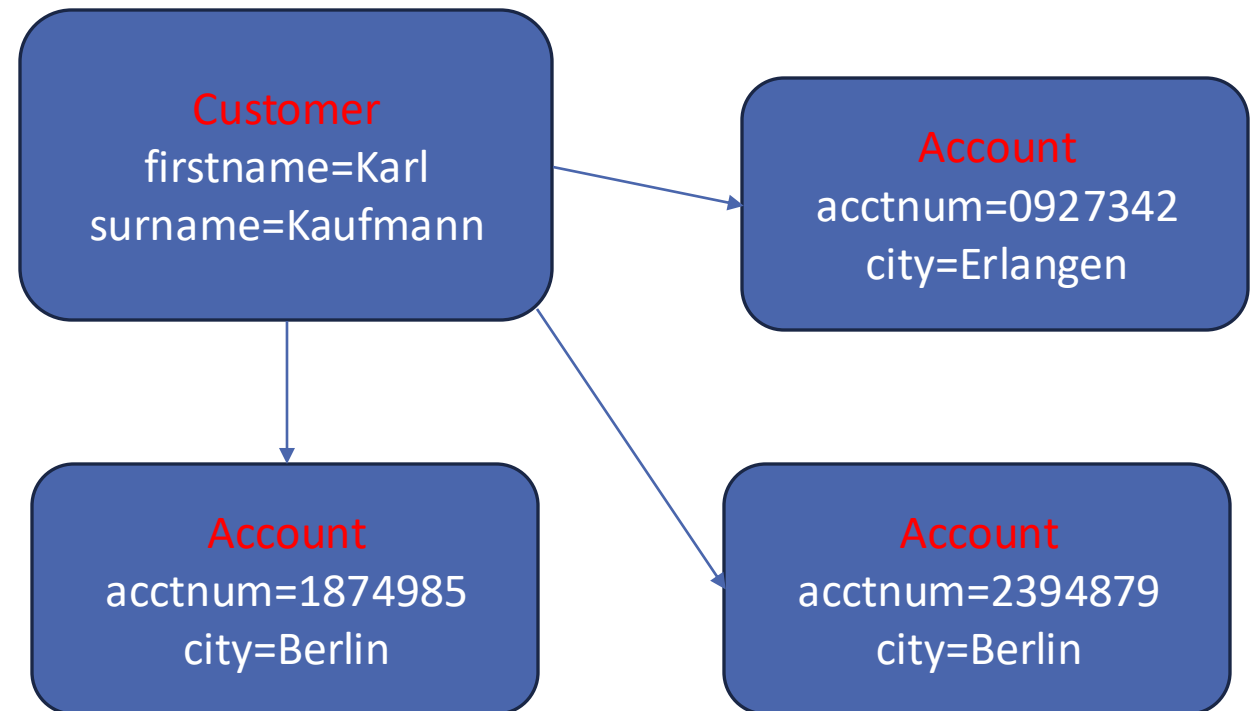
Branch

id	city
6	Berlin
9	Erlangen

Account

id	acctnum	branch	customer
1	1874985	6	2
2	2394879	6	2
3	0927342	9	2

Object-Oriented Design



JDBC – Java Database Connectivity

Included in JDK

Application programming interface (API) for connecting to databases

Requires a driver for the intended database

- Download the appropriate driver
- Add .jar to classpath

Steps for Connecting to a Database with Java

Step 1: Create a connection

Step 2: Create a statement

Step 3: Execute the query

Step 4: Process the retrieved results

Step 5: Close the connection and release resources

Need multiple try/catch statements with SQLExceptions in case any stage fails

Step 1: Create a connection

Previous versions required driver registration

- From JDBC 4.0 and onwards, drivers in the classpath are automatically loaded

Three necessary Strings:

- Database url: location and name of the database
- Username: designated database user (root, etc.)
- Password: corresponding account password

Example:

```
Connection myConnect = DriverManager.getConnection("jdbc:postgresql://localhost/pets", "oop", "ucalgary");
```


Step 2: Create a statement

Three statement types

- Statement: Simple SQL statement with no parameters
- PreparedStatement: SQL statements with input parameters
- CallableStatement: used for stored procedures with input/output parameters

Statement:

```
Statement myStmt = dbConnect.createStatement();
```

PreparedStatement:

```
String query = "INSERT INTO cat (name, owner, birth) VALUES (?, ?, ?)";
```

```
PreparedStatement myStmt = dbConnect.prepareStatement(query);
```

```
myStmt.setString(1, name);
```

```
myStmt.setString(2, owner);
```

```
myStmt.setDate(3, birthdate);
```

Step 3: Execute the statement

Three execute options

- `execute`: returns true if first object returned by the query is a `ResultSet` object
- `executeQuery`: returns one `ResultSet` object
- `executeUpdate`: returns an integer value for the number of rows affected by the statement (needed for transactions that update, insert, or delete)

Statement:

```
ResultSet results = myStmt.executeQuery("SELECT * FROM cat");
```

Prepared Statement:

```
int rowCount = myStmt.executeUpdate(); //after preparing an insert
```

Step 4: Process the retrieved results

A `ResultSet` object stores the results of a query

Allows the data results to be used within an application

<https://docs.oracle.com/javase/tutorial/jdbc/basics/retrieving.html>

```
try{
    Statement myStmt = dbConnect.createStatement();
    results = myStmt.executeQuery("SELECT * FROM cat");

    while (results.next()){
        System.out.println("Print results: " + results.getString("name") + ", " +
            results.getString("owner"));

        catsAndOwners.append(results.getString("name") + ", " + results.getString("owner"));
        catsAndOwners.append('\n');
    }
} catch (SQLException ex) {
    ex.printStackTrace();
}
```

Step 5: Close the connection and release resources

Good practice to release resources

Close all statement objects after use

Close `ResultSet` objects

Close `Connection` objects

Use built-in `close()` methods

Exercise 26.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise 26.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Java Database Connectivity (JDBC)

Retrieval of information from a database using Java

Modifying a database using Java

Relational databases versus object-oriented programming

Summary: Skills

Using JDBC to access a PostgreSQL database

Creating a database connection

Creating and executing statements

Creating and executing prepared statements with input parameters

Using retrieved data

Releasing resources upon completion

Additional Information

Related reading

- Volume 2, Chapter 5: Database Programming